AXI Read DMA v1.0
Formal Specification

Read Only - INCR Burst - Aligned Access - Single Outstanding - 4KB Safe

目标

从 AXI Memory 读取连续数据，通过 AXI4 Read burst 接收 RDATA，并顺序写入片上 SRAM。第一版优先保证协议正确、结构清晰和易验证。

1. v1.0 Scope

支持：AXI4 Read、INCR burst、aligned address、length 为 beat bytes 整数倍、最大 256 beats/burst、4KB boundary split、single outstanding、RRESP 检查、SRAM 顺序写入。

暂不支持：Write DMA、Descriptor、Scatter-Gather、2D stride、unaligned、WRAP、multiple outstanding、多个 AXI ID。

2. System Context

```
DRAM / Memory
↓
Memory Controller / AXI Interconnect
↓ AXI4 Read
AXI Read DMA v1.0
↓
A/B SRAM
↓
4x4 Systolic Array
```

3. Command Interface

Signal	Dir	Meaning
clk	in	Clock
rst_n	in	Active-low reset
start	in	Start one transfer
src_addr	in	Source byte address
length_bytes	in	Total bytes
busy	out	DMA active
done	out	Transfer complete pulse
error	out	Parameter or AXI error

4. AXI Read Interface

Signal	Dir	Meaning
ARADDR	out	Burst start address
ARLEN	out	Burst beats - 1
ARSIZE	out	log2(bytes per beat)
ARBURST	out	INCR
ARVALID	out	AR request valid
ARREADY	in	AR request accepted
RDATA	in	Read data
RRESP	in	Read response
RLAST	in	Last beat
RVALID	in	R payload valid
RREADY	out	DMA ready for R beat

核心 handshake：ARVALID && ARREADY 接受 read request；RVALID && RREADY 接受一个 data beat。

5. Burst Manager

```
BytesPerBeat = 2^ARSIZE
MaxBurstBytes = 256 x BytesPerBeat
BytesTo4K = 4096 - (current_addr mod 4096)

BurstBytes = min(remaining_bytes, MaxBurstBytes, BytesTo4K)
BurstBeats = BurstBytes / BytesPerBeat
ARLEN = BurstBeats - 1
```

每完成一个 burst：current_addr += burst_bytes；remaining_bytes -= burst_bytes。remaining_bytes==0 才表示整个 DMA command 完成。

6. Example: 64-bit AXI, 4096-byte Read

DATA_WIDTH=64 bit -> 8 bytes/beat -> ARSIZE=3。4096 bytes 共 512 beats，因此至少拆成两个 256-beat INCR bursts。

```
Burst 0: ARADDR=0x8000_0000, ARLEN=255, bytes=2048
Burst 1: ARADDR=0x8000_0800, ARLEN=255, bytes=2048
```

7. 4KB Boundary Example

current_addr=0x8000_0F00，remaining=2000 bytes。到下一个 4KB boundary 只有 256 bytes，所以第一 burst 只能是 256 bytes，即 32 beats，ARLEN=31。下一 burst 从 0x8000_1000 开始。

8. Control FSM

```
IDLE
↓ start
CHECK_CMD
↓
CALC_BURST
↓
SEND_AR
↓ ARVALID && ARREADY
RECV_R
↓ RVALID && RREADY && RLAST
UPDATE
 ├- remaining != 0 -> CALC_BURST
 └- remaining == 0 -> DONE
```

9. Data Path to SRAM

```
RDATA / RVALID
↓
AXI Read Engine
↓
FIFO
↓
SRAM Writer
↓
sram_we / sram_addr / sram_wdata
```

FIFO 满时 RREADY=0，形成 backpressure。beat counter、FIFO pointer、SRAM address 都只能在 RVALID && RREADY 时推进。

10. Accelerator Example

Object	Type	Bytes	64-bit AXI beats
A tile	4x4 INT8	16	2
B tile	4x4 INT8	16	2
C tile (future Write DMA)	4x4 INT32	64	8

读取 packed A tile：length=16 bytes，ARSIZE=3，BurstBeats=2，ARLEN=1。v1.0 先使用 packed contiguous layout，不处理普通大矩阵中的 stride。

11. Verification Plan

Test	Purpose
1 beat	Minimum legal transfer
256 beats	Maximum burst
257 beats	Split 256 + 1
4096 bytes	Multiple bursts
Near 4KB boundary	Boundary split
Random ARREADY stall	AR payload stability
Random RVALID gaps	Variable memory latency
RREADY stall	Backpressure
RRESP error	Error path
Bad alignment/length	Command checker

关键 checks：ARVALID&&!ARREADY 时 AR payload 稳定；ARLEN<=255；burst 不跨 4KB；accepted beat count=ARLEN+1；最后一 beat 对应 RLAST。

12. Recommended RTL Files

```
rtl/
|- axi_read_dma_top.sv
|- dma_read_ctrl.sv
|- dma_burst_manager.sv
|- axi_read_engine.sv
|- dma_fifo.sv
`- sram_write_adapter.sv
```

推荐实现顺序：Burst Manager -> AR Engine -> R Engine -> SRAM Writer -> FIFO/backpressure -> Top integration。